AUTO CDC INTO – Streaming CDC Cheat Sheet

A working Lakeflow Spark Declarative Pipeline (SDP) that shows every AUTO CDC option on small sample data you can follow row by row. Release statuses were checked against docs.databricks.com on 2026-10-23. Check them again before you reuse this.

Files

File	What it is
`00_setup_sample_data.py`	Notebook. Creates the schema, volume, sample CDC files (batches 1–4) and the source tables.
`transformations/python/auto_cdc_examples.py`	Pipeline source in **Python**, covering Examples 0–11.
`transformations/sql/auto_cdc_examples.sql`	The same pipeline in **SQL**. It leaves out the Python-only examples 8, 9, 10 and 11b.
`03_dbsql_standalone_auto_cdc.sql`	AUTO CDC in Databricks SQL with **no pipeline** (Beta).
`04_explore_results.sql`	Queries to inspect each target, with the expected results.

Run it

1. Run `00_setup_sample_data.py` and set the catalog and schema widgets. It lands **batch 1**.
2. Create an ETL pipeline. Set the default catalog and schema to the ones from step 1, and set the source to either the Python file **or** the SQL file. Add two configuration values: `cdc_demo.source_catalog` and `cdc_demo.source_schema`. Use **Triggered** mode and serverless (or the Pro/Advanced edition).
3. Run the pipeline, then run `04_explore_results.sql`.
4. Run `land_batch(2)`, then run the pipeline and explore again. Repeat for batch 3, and run batch 4 (TRUNCATE) last.

Syntax at a glance

SQL (inside a pipeline)

```
CREATE OR REFRESH STREAMING TABLE customers_current;          -- 1. declare target

CREATE FLOW customers_current_flow AS AUTO CDC INTO customers_current
FROM STREAM(customers_cdc_clean)                                -- 2. streaming source
KEYS (customer_id)                                              -- required
IGNORE NULL UPDATES                                             -- optional: partial updates
APPLY AS DELETE WHEN operation = 'DELETE'                       -- optional
APPLY AS TRUNCATE WHEN operation = 'TRUNCATE'                   -- optional, SCD1 only
SEQUENCE BY sequence_num                                        -- required (STRUCT(a, b) for ties)
COLUMNS * EXCEPT (operation)                                    -- optional
STORED AS SCD TYPE 2                                            -- 1 (default) | 2 | BITEMPORAL (Beta)
TRACK HISTORY ON * EXCEPT (last_login_ts);                      -- optional, SCD2
```

Python

```python
from pyspark import pipelines as dp
from pyspark.sql import functions as F

dp.create_streaming_table("customers_history")
dp.create_auto_cdc_flow(
    target="customers_history", source="customers_cdc_clean",
    keys=["customer_id"], sequence_by=F.col("sequence_num"),
    apply_as_deletes=F.expr("operation = 'DELETE'"),
    except_column_list=["operation"],
    stored_as_scd_type=2,
    track_history_except_column_list=["last_login_ts"],
)

# Source has only full copies? Diff snapshots instead (Python only):
dp.create_auto_cdc_from_snapshot_flow(
    target="customers_from_snapshot", source="catalog.schema.customers_full_snapshot",
    keys=["customer_id"], stored_as_scd_type=2,
)
```

SCD2 targets get `__START_AT` / `__END_AT` (type = SEQUENCE BY type); `__END_AT IS NULL` = current row.

What each example teaches

#	Example	Option(s) shown	Watch for
0	Bronze plus a cleaned view	Auto Loader / `read_files`, expectations	Rows with a NULL key or NULL sequence are dropped *before* AUTO CDC
1	SCD Type 1	`keys`, `sequence_by`, `apply_as_deletes`, `apply_as_truncates`, `except_column_list`, tombstone retention	Out-of-order events (104) and late events (101) still resolve correctly
2	SCD Type 2	`stored_as_scd_type=2`, `track_history_except_column_list`	`__START_AT` / `__END_AT` columns. A login alone doesn't create a new version
3	Narrow target	`column_list` / `COLUMNS a, b, c`	Allow-list versus deny-list
4	Composite key	`keys=[a, b]`, `sequence_by=struct(ts, id)`	Ties on the timestamp are broken by `event_id`
5a	Partial updates	`ignore_null_updates` (plus the `_column_list` / `_except_column_list` variants)	A NULL means "no change"
5b	Partial updates	`columns_to_update` / `COLUMNS TO UPDATE`	Only the columns each event lists are written
6	Multiple flows, one target	`name`, `once=True` / `AUTO CDC ONCE`	A legacy back-fill runs once, then the live feed takes over
7	Bitemporal (Beta)	`system_sequence_by`, `STORED AS BITEMPORAL`	Commented out. Needs the PREVIEW channel
8	Delta Change Data Feed source	`readChangeFeed`, `_change_type`, `_commit_version`	Drop the `update_preimage` rows
9	Snapshot table	`create_auto_cdc_from_snapshot_flow(source="table")`	A key missing from the next snapshot counts as a delete
10	Versioned snapshots	`source=<function>` returning `(df, version)` or `None`	The first call must return a snapshot

#	Example	Option(s) shown	Watch for
11	Downstream	Materialized view on `__END_AT IS NULL`, plus streaming the target's change feed	Never read an AUTO CDC target as a plain append stream

AUTO CDC vs. AUTO CDC FROM SNAPSHOT vs. MERGE INTO

	AUTO CDC	AUTO CDC FROM SNAPSHOT	Hand-written MERGE INTO (in a Job)
Use when	The source emits change events (Debezium, Lakeflow Connect, app events, CDF)	The source only gives full copies or periodic extracts	You need logic AUTO CDC can't express, or you're outside SDP
Out-of-order / late data	Handled automatically through SEQUENCE BY	Handled by snapshot version order	You build it (dedupe, windowing, watermarks)
SCD2 history	One option: `STORED AS SCD TYPE 2`	Yes	Lots of custom code
Deletes	`APPLY AS DELETE WHEN`, with tombstones for late events	Inferred when a key is missing	You build it
Cost	Incremental, processes only new events	Reads and diffs the whole snapshot each time, so cost grows with table size	Depends on how you write it
Languages	Python, SQL, DBSQL standalone (Beta)	Python only (in SDP)	Any

Pros: Far less code. Ordering, late data, deletes and SCD2 are all handled correctly. Expectations, lineage and the event log come built in. **Cons:** It needs a sortable sequence column with no NULLs. SCD2 can't truncate. You can't read the target as an append-only stream. It needs serverless or the Pro/Advanced edition. Custom per-row merge logic isn't possible.

Release status (as of 2026-10-23)

Feature	Status
AUTO CDC INTO / `create_auto_cdc_flow` (replaces APPLY CHANGES / `apply_changes`)	GA
AUTO CDC FROM SNAPSHOT / `create_auto_cdc_from_snapshot_flow`	GA in the docs (no preview badge). Python only
Partial updates (`IGNORE NULL UPDATES ON …`, `COLUMNS TO UPDATE`)	GA (announced Aug 2026)
Bitemporal (`STORED AS BITEMPORAL`, `SYSTEM SEQUENCE BY`)	Beta (PREVIEW channel)
Standalone AUTO CDC in Databricks SQL	Beta (DBR 17.3+, serverless SQL warehouse)
AUTO CDC in open-source Apache Spark	In progress. SQL interface merged for the next Spark release

Gotchas for new engineers

- Declare the target with `create_streaming_table` / `CREATE OR REFRESH STREAMING TABLE` **before** you add the flow.
- The source must be a **stream** (`STREAM(...)` / `readStream`). The one exception is a `ONCE` flow, which reads a batch.

- Make sure every type is right: SEQUENCE BY types must match across all flows that write to the same target.
- A plain SCD1 target overwrites columns with NULLs when the source sends partial rows. Use 5a or 5b to avoid this.
- Any target that doesn't define a truncate rule must have TRUNCATE rows filtered out of its source.
- Delete tombstones are kept for 2 days by default (`pipelines.cdc.tombstoneGCThresholdInSeconds`). Raise this if your late data can arrive later than that.
- A full refresh rebuilds the target from the source. Make sure the source still holds all of its history first.

Rearc · Training Resources for Data Engineers · Release status checked 2026-10-23 against docs.databricks.com — recheck before reuse. Companion code: “AUTO CDC Examples” folder.